

Fused MoE GEMM 算子优化技术方案

赛题：XH-202608 基于国产软件栈大模型推理前沿算子优化（TileLang 赛题）

评测账号：muxi2026C1050 | 最终成绩：XPU-OJ Accepted 80.33 (2026-09-05, submissionId 139770, timeUsed 16 031 μ s 历史最快；三点 81/80/80)

1. 问题定义

实现 DeepSeek 风格 MoE 的 pre-routed fused expert GEMM：对已按 expert 分组的 token，完成 gate/up 两路 GEMM、SiLU 激活、down GEMM 与路由加权，输出写回评测方给定的 padding 行序。评测在 MetaX C500 GPU 上进行，计分公式为每用例 $100 \cdot s / (1 + s)$ (s = 基线用时/内核用时)，三用例取均值——因此三个用例必须同时优化。

用例	n_experts	d_hidden	d_expert	group_sum	warmup/iters
case1	16	2048	8192	2272	5 / 30
case2	32	7168	2048	4544	5 / 20
case3	64	7168	2048	9088	5 / 20

合规约束：GPU 计算算子仅可使用 TileLang；禁止 PyTorch 计算、禁止 import_source/call_external 注入或调用外部设备实现、禁止异步与流水线语义、禁止跨调用缓存结果、禁止 mcTlass 等外部库。本方案全部实现与结论均在上述边界内。

2. 硬件与软件环境

组件	配置（实测试验环境）
GPU	MetaX C500, 64GB 显存；实测 DRAM 峰值带宽约 1.44 TB/s, L2 约 8 MB, warp = 64 线程
MACA 软件栈	3.7.x / 3.8.x（驱动 3.8.30）
编译器	TileLang-MetaX 0.1.x（race 分支）
框架	torch 2.8.0+metax（仅 host 侧张量分配）、Python 3.12

3. Kernel 总体设计（v432 最终形态）

3.1 两阶段结构

Stage1 (fused Gate+Up)：每个网格点一次加载 A 块（token 块），先后与 gate/up 权重块做两次 MMA (fp32 累加)，随后在 epilogue 融合 SiLU 激活并写 workspace (up_logits)。Stage2 (Down)：以 up_logits 为输入做下投影 GEMM，逐行乘路由权重写 out，padding 行显式清零。两阶段同流先后 launch，依赖关系由流序天然保证。

3.2 分块与网格

参数	Stage1	Stage2	说明
M 块 (token)	128	128	与评测方 group_idx_for_bx 粒度一致
K 块	64	64	hidden / intermediate 方向
N 块	128	128	intermediate / hidden 方向
threads	256	256	4 warp $\times$ 64 lane
MMA 策略	Square	Square	六维扫描中的最优

3.3 关键机制

- (a) A 操作数 shared 化：MMA 的 A (token) 操作数由 fragment 改为 alloc_shared，消除 LDS→寄存器的分散搬运，纯 GEMM 吞吐由 26.8 提升至 87 TFLOPS (3.2 倍)，为全方案最大单点收益；gate/up 两次 MMA 复用同一份 A 块，A 的全局加载次数减半。
- (b) gate/up 记分板重叠 (up 寄存器预取)：up 权重块提前加载至 fragment 寄存器，在 gate MMA 执行期间完成全局加载，gate MMA 结束后仅做寄存器→shared 短途搬运即执行 up MMA。该重叠不使用任何硬件异步指令，仅依赖加载指令的记分板机制，因此满足“禁用异步”的合规边界；hidden=7168 档实测正收益，case1 分派全开后 -6.8%。
- (c) 编译 pass 开关组合：tl.enable_fast_math (全部 kernel，-0.5~1% 三 case 一致正收益)、tl.enable_lower_ldgstg_predicated (谓词化全局加载 lowering)、tl.disable_safe_memory_legalize 与 tl.disable_vectorize_256 (仅 Stage2-fast，逐字节等价对拍后启用)、TL_DISABLE_WARP_SPECIALIZED (MACA 后端必需)。
- (d) 访存合并：全部权重 T.copy 指定 coalesced_width=8 (16 字节/线程)，实测优于默认值；更大宽度越界 (已实测排除)。
- (e) Stage2 满块快路径 epilogue：满块 (actual_rows == 128) 走无谓词写回，尾块保留谓词；padding 行显式写 0 保证语义一致。
- (f) MMA k_pack：hidden ≥ 7000 时 Stage1 使用 k_pack=2 (单条 MMA 打包 2×16 的 K 切片)，case1 保持 1；均经扫描验证。
- (g) JIT 编译缓存与 workspace：_KERNEL_CACHE 缓存编译结果 (非数据)；up_logits 为 scratch 缓冲，每次调用重新计算填充，符合题目“建议使用 up_logits 作为中间缓冲区”的指引。

3.4 per-shape 分派 (分派全开)

hidden ≥ 7000 (case2/3) 与 hidden ≤ 4096 (case1) 在 K 循环长度与权重访存占比上差异显著。v432 对全部 case 统一启用增强 kernel (stage1_prefetch + stage2_fast)：case1 -6.8%、case2 -2.9%、case3 -3.7%。分派依据仅为输入 shape (kernel 特化)，不涉及结果缓存或阶段区分。

4. 技术路线与关键实验

4.1 演进主线 (OJ 实测)

版本	关键变更	OJ 分数
v6	官方模板调参基线	72.67
v138	A-shared + 权重 cw4 + column swizzle	75.67
v226	Stage1 MMA warp 策略 Square	76
v278	权重 copy coalesced_width 4→8	76.33
v282/v293	Stage2 column swizzle (两连 Accepted)	76.67
v404	Stage2 Down next-K 同步预取 (hidden=7168)	78.00
v412	per-shape 分派体系成形	78 档
v432	+fast_math +ldgstg_predicated +分派全开	78.67 (rank 27)
v469	Stage2 手写同步调度 (去 Pipelined) + MMA swizzle 布局 vec4	79.33
v478	Stage1 双侧 vec4 布局 + Up 加载 cw8 + panel2 + k_pack=2 全开	79.67
v718	+ E64 Stage1 terminal-K builder (per-shape 特化)	80.33 (139753)

v719	+ E64 Stage2 双B emitter	80.33 (139764, 用时更短)
v720	+ E16 Stage2 双B emitter 分派扩展	80.33 (139770, 16 031 μ s 历史最快)

4.2 已验证的负结果

- T.Pipelined num_stages $\geq$ 2: MACA 无 cp.async, 多缓冲只剩同步开销, 三次独立实验均 -20% 以上;
- CUDA 流/event 跨 kernel 重叠: MACA 运行时不遵守跨流 event 依赖, 产生数据竞态 (4 次复现, 含基线同窗 Accepted 对照);
- M256 大 M 块合并 / block_M=64: 寄存器压力或权重重读放大, 均负收益;
- thread=512、bh=32/128、be=64、swizzle panel=8、row 序、k_pack=4: 实测更慢或编译失败;
- int8 量化 + 跨调用复用: 违反合规禁令且精度不匹配, 未采用;
- 手写原生 MFMA (import_source 注入): 后被官方禁令明确排除, 相关历史探针弃用。
- 官方 TensorCoreIntrinEmitter 严格同步微探针: 128 $\times$ 128 $\times$ 2048 微基准 0.184 ms, 比同形状朴素 T.gemm (0.242 ms) 快 24%, 但集成进 MoE 后全部无提分 (v487-v490 四变体) ——MoE 已受 global/LDS 搬运主导, MMA 生成质量不是剩余瓶颈 (emitter 路线终审关闭);

4.3 方法论

评测机速度存在 $\pm 50\%$ 档位波动, 一切优化结论以同窗口对照为准; 候选先经本地 judge 同构 harness 的随机 seed 认证 (32 组覆盖三 case 与边界形状, bad=0 才上 OJ), 再以两次连续 Accepted 确认。

5. 创新点

- A 操作数 shared 化: MMA 操作数布局从 fragment 迁移到 shared, 纯 GEMM 吞吐 3.2 倍 (26.8 $\rightarrow$ 87 TFLOPS);
- 无异步指令的 load/MMA 重叠: 基于 MACA 记分板语义的同步寄存器预取, 在禁用异步的约束内实现传统依赖 cp.async 的拷贝-计算重叠;
- 编译 pass 开关组合调优: 四个 pass 开关的逐 kernel 组合, 均经逐字节等价性对拍后启用;
- per-shape kernel 特化体系: 按输入 shape 选择 kernel 构造器与 pass 组合;
- 系统化合规实验方法: judge 同构 harness + 随机 seed 认证 + 同窗 OJ 对照, 全部结论可复现、可审计。

6. 结果

指标	数值
OJ 最高分	80.33 (Accepted, 三点 81/80/80), submissionId 139770 (2026-09-05, 16 031 μ s 历史最快; v718/v719/v720 三份独立代码均 ≥ 80.33)
OJ 三点用时	总 16 031 μ s (v720); SPJ score ratio 0.814/0.801/0.802
本地 C500 实测	2.789 / 5.790 / 9.137 ms (warmup10 / iters100)
功能测试	7 个形状 (含单 expert、valid=127 非整块、hidden=128 边界) 全部通过
正确性	对官方 fp32 参考语义逐元素对拍 bad=0; 32 组随机 seed 零失败
显存	23.27 GB (评测机 64 GB 内)

独立参照验证：以题目公告给出的 fp32 参考实现为参照，case4/5/6 逐元素对拍 bad=0 ($\max_abs \leq 0.0156$ ，远小于 $\text{rtol/atol}=0.05$)。

6.1 性能分解与瓶颈分析（实测）

阶段	case1	case3	瓶颈类型
Stage1 (fused Gate+Up)	≈2.0 ms (72%)	≈7.1 ms (78%)	MMA 吞吐受限：87 TFLOPS 上限的 70%
Stage2 (Down)	≈0.8 ms (28%)	≈2.0 ms (22%)	访存受限：≈100% DRAM 带宽

Stage1 提升受限于 TileLang 可达的 MMA 供给上限（87 TFLOPS，操作数经 shared 的同步供给路径）；Stage2 已贴近带宽上限。两者性质不同，因此优化手段分别针对 MMA 效率（k_pack / warp 策略 / 寄存器预取）与访存合并（coalesced_width / 满块 epilogue）——与官方答疑推荐的先判瓶颈类型再调优的方法论一致（issue 50）。

7. 合规性声明

官方答疑（GitLink issue 50）明确：发榜单位会检查成功提交的代码，异步拷贝一经发现成绩作废——本方案最终代码经静态扫描与人工审计确认不含任何此类实现。

最终代码仅使用 TileLang 语言原语（T.Kernel / T.copy / T.gemm / T.Parallel / T.alloc_shared / T.alloc_fragment / T.Pipelined(num_stages=1)）与官方编译 pass 开关；未使用 PyTorch 计算算子、未注入外部设备代码、未调用外部设备函数、未使用任何异步/流水线语义、未进行跨调用结果缓存、未使用 mcTlass 等外部库。torch 仅用于 host 侧工作区张量分配（与官方参考模板一致）。

8. 复现指引

见提交包
source/README.md（环境依赖、功能测试、性能测试完整命令）；优化全过程与全部正/负结果见 logs/OPTIMIZATION_LOG.md（1000+ 条实验记录）；评测平台提交记录见 logs/oj_submission_records.json。